Software Development Report

Project Title Placeholder

Brandon MacGillivray

Student ID Placeholder

April 21, 2026

Acknowledgements

Contents

1 Introduction **2**

2 Purpose, Operation and Validation **2**

3 Design and Implementation **2**

3.1 Significant Assumptions Made for the Design and Implementation 3
3.2 Languages, Packages, Frameworks, and Libraries Used 5
3.3 Overall Architecture / System Design and Interfaces Between Components . . . 6
3.4 Data Model(s), Including In-Memory Structures and Persistent Storage 9
3.5 Significant Methods, Functions, or Other Lower-Level Elements 9
3.6 Important Errors and Their Conditions 11
3.7 User Interface Design 12
3.8 Quality Assurance Steps 12
3.8.1 Version Control and Versioning 13
3.8.2 Code Style and Convention 14
3.8.3 Automated Testing 14

A Appendices **15**

1 Introduction

Hand pose estimation is an important computer-vision task for touchless human-computer interaction, especially in edge-oriented settings such as AR/VR, public displays, and other real-time interactive systems where accuracy must be balanced against limited compute resources. This project focuses on lightweight 2D RGB hand pose estimation and implements a DRHand-inspired dual-branch approach in a form suitable for training, evaluation, inference, benchmarking, and reproducible experiment reruns.

The software artefact is structured as a research-oriented repository rather than an end-user application. It combines a reusable Python package for the core hand-pose pipeline with command-line scripts, experiment definitions, preserved checkpoints, and supporting documentation. This report describes the purpose and operation of the software, its design and implementation, the main architectural components and interfaces, the data representations it uses, and the quality-assurance steps taken during development.

2 Purpose, Operation and Validation

- What is the software intended to do?
- How does a user run or operate it?
- What inputs does it take and what outputs does it produce?
- How can the user tell it is working correctly?
- What evidence shows it performs its intended purpose?

The software is intended to implement and evaluate a lightweight 2D hand pose estimation pipeline for edge HCI, with a DRHand-inspired dual-branch model that predicts hand keypoints from RGB images and supports training, inference, benchmarking, and reproducible experiment reruns.

A user operates the software from the command line by setting up the documented Python environment, then running the repository scripts for the required workflow, such as `scripts/train.py` for training, `scripts/eval_metrics.py` for checkpoint evaluation, and `scripts/predict_image.py` for single-image inference, with inputs and options supplied as command-line arguments.

The software takes inputs such as dataset roots, checkpoint files, single RGB images, model and evaluation settings passed through command-line arguments, and it produces outputs such as trained checkpoint files, evaluation JSON metrics, aggregated CSV result tables, benchmark summaries, and qualitative prediction overlays or rendered figures.

A user can tell the software is working correctly when the lightweight test suite passes, the main command-line scripts load and print valid help output, and training, evaluation, inference, or benchmarking workflows execute successfully and produce outputs in the expected format that can be compared against the preserved reference JSON, CSV, checkpoint, and qualitative artefacts bundled in the repository.

3 Design and Implementation

- How is the software designed overall?
- How is that design implemented in code?
- What are the main parts of the system and how do they work together?

The software is designed as a modular research artefact with a reusable Python package under `src/` for the core hand-pose pipeline, a set of command-line scripts under `scripts/` for running workflows such as training, evaluation, inference, and benchmarking, experiment definitions under `experiments/`, and curated documentation and preserved artefacts under `docs/`, allowing the same core implementation to support both development and reproducible assessment workflows.

The overall design is implemented as a structured Python package in which dataset handling is separated under `handpose.data`, model architecture and losses are defined under `handpose.models`, reusable training logic is kept under `handpose.training`, evaluation logic under `handpose.evaluation`, inference and fusion logic under `handpose.inference`, and checkpoint persistence under `handpose.checkpoints`, while thin command-line scripts call these modules to run complete workflows.

The main parts of the system are the dataset layer, the DRHand-inspired model, the inference and fusion layer, the training and evaluation pipelines, checkpoint management, and the command-line workflow scripts, and they work together by loading image and annotation data into a consistent format, passing that data through the model to generate heatmap and coordinate predictions, optionally fusing those predictions for final output, saving or loading checkpoints to preserve model state, and exposing these capabilities through scripts for training, evaluation, benchmarking, qualitative rendering, and single-image prediction.

3.1 Significant Assumptions Made for the Design and Implementation

- What key assumptions were made when designing the software?
- How do those assumptions affect how the software works or is used?

Framework equivalence. The software assumes that the DRHand method can be reimplemented in PyTorch even though the original paper describes a TensorFlow implementation. As a result, this repository should be understood as a DRHand-inspired reconstruction rather than a strict reproduction, so exact numerical agreement with the original paper should not be expected.

Dataset substitution. The software assumes that the dual-branch DRHand idea can be studied meaningfully on RHD, HK26K / `coco_hand`, and RH8 even though the original paper used YouTube2D Hands, GANerated Hands, STB, and RHD. This means the software evaluates the method under a different data regime, so its results support the reconstructed pipeline used in this project rather than the original paperâ€™s exact benchmark setup.

Prepared input assumption. The software assumes that each input image already contains a clearly visible hand region and therefore does not implement a separate hand detector or crop-normalisation stage. This affects use because users must provide images or datasets where the hand is already reasonably framed, otherwise performance may degrade when the full image is simply resized to the network input size.

Fixed preprocessing strategy. The software assumes that full-image resize to 256 x 256 is an acceptable preprocessing strategy because the paper specifies the input size but does not define a more detailed input-normalisation pipeline. This makes the system simpler and easier to reproduce, but it also means the model is sensitive to changes in hand scale, position, and framing within the input image.

Coordinate representation. The software assumes that keypoints are represented internally as normalized (x, y) coordinates in the resized image space. This affects how the system works because training, inference, fusion, and evaluation all depend on this normalized representation,

and conversion back to original pixel coordinates is only done when exporting predictions for human inspection or downstream use.

Heatmap supervision reconstruction. The software assumes Gaussian target heatmaps for heatmap regression because the paper defines an L2 heatmap loss but does not specify exactly how target heatmaps are constructed. This affects optimisation because the heatmap branch is trained to match smooth Gaussian peaks rather than some other target representation, which changes convergence behaviour and localisation sharpness.

Heatmap width assumption. The baseline system assumes `heatmap_sigma = 2.0` because the original paper does not state a fixed Gaussian width. This matters because the spread of the target heatmap changes the difficulty and smoothness of the heatmap prediction task, so reproducing results depends on keeping this value consistent.

Wing-loss parameter assumption. The software assumes baseline Wing-loss values of w = 10.0 and epsilon = 2.0 because the paper gives the formula but not the actual parameter settings. This directly affects how the coordinate branch is trained, since different values change the relative penalty for small, medium, and large localisation errors.

Stage-2 optimisation assumption. The software assumes that stage 2 should jointly optimise both the heatmap and coordinate branches after the coordinate branch is added. This means the second training stage is not treated as coordinate-only fine-tuning, and the final model behaviour reflects continued interaction between both branches during optimisation.

Loss-weighting assumption. The baseline implementation assumes equal stage-2 loss weights for heatmap and coordinate supervision because the paper does not specify how the two losses should be balanced. This affects performance because the default behaviour gives both branches equal influence, while changing these weights shifts training emphasis toward one branch or the other.

Parameter update policy. The default implementation assumes that the backbone and heatmap branch remain trainable during stage 2 unless freezing is explicitly requested. This affects how the model learns because the full network can continue adapting in the second stage instead of treating stage-1 features as fixed.

Effective batch-size reconstruction. The software assumes that the paperâ€™s stage-2 batch size of 256 can be approximated using batch size 64 with gradient accumulation on different hardware. This makes the training pipeline usable on more limited systems, but it also means the repository reproduces the paperâ€™s intended effective batch size only approximately rather than literally.

Convergence assumption. The paper states that stage 1 runs until convergence and mentions 11 epochs in its original setup, but it does not define a complete stopping rule for this implementation context. The software therefore assumes a larger epoch budget together with early stopping, which means training duration is reconstructed pragmatically rather than copied exactly from the paper.

No explicit augmentation policy. The software assumes a simple deterministic preprocessing pipeline because the paper does not specify a detailed augmentation strategy. This improves clarity and reproducibility, but it also means robustness is achieved without relying on a strong augmentation regime unless a user adds one manually.

Single-hand target assumption. The software assumes one target hand per sample, selecting the requested or most visible hand where needed rather than jointly predicting multiple hands. This affects use because the repository is designed for single-hand pose estimation workflows rather than general multi-hand scene understanding.

Fusion threshold definition. The paper says fusion uses the median predicted knuckle length as the adaptive threshold alpha, but it does not fully formalise the exact skeleton edges in text. The implementation therefore assumes explicit edge lists for supported skeleton layouts, which means the behaviour of fusion depends partly on this code-level reconstruction.

Reduced 10-keypoint extension. The software assumes that the same overall DRHand-style architecture can be reused for a reduced 10-keypoint fingertip-and-base layout even though the original paper is written for 21 keypoints. This allows the project to test whether a smaller output space is sufficient for edge-HCI use, but it also means the reduced model is an adaptation introduced by this project rather than something directly defined in the paper.

Reduced-layout fusion assumption. Because the original paper does not define how fusion should work for a reduced keypoint layout, the implementation assumes a simplified edge set for the 10-keypoint model. This affects behaviour because the reduced fusion rule is a project-specific extension and therefore its outputs depend on an additional design decision beyond the original paper.

Visibility-aware evaluation assumption. The software assumes that evaluation should mask invisible joints when visibility information is available. This affects the reported metrics because performance is measured primarily on visible landmarks, which may differ from alternative evaluation protocols.

Reduced-layout EPE root assumption. The reduced shared-10 layout does not include the same root structure as the full 21-keypoint skeleton, so the implementation assumes an alternative root for root-relative EPE when necessary. This keeps EPE usable for reduced-layout experiments, but it means that reduced-layout EPE is not a perfectly identical measurement setup to the full-layout case.

3.2 Languages, Packages, Frameworks, and Libraries Used

- What technologies were used to build the software?
- Why were these choices used in this project?

Python. The software was built primarily in Python because it provides a practical and widely used language for machine-learning research, rapid experimentation, data processing, and command-line workflow development. This made it suitable for implementing a research artefact that needed to support training, evaluation, inference, benchmarking, and reproducibility in a consistent codebase.

PyTorch. The main hand-pose pipeline was implemented in PyTorch because the project required a flexible deep-learning framework for defining custom model architectures, losses, and training procedures. PyTorch was particularly suitable for this DRHand-inspired implementation because it allowed the original dual-branch idea to be reconstructed clearly while remaining easy to debug and adapt for the reduced 10-keypoint and transfer-learning experiments.

Conda. A Conda-managed environment was used because the project depended on a mix of machine-learning, image-processing, and benchmarking packages that needed to be installed consistently across systems. Conda was chosen to reduce dependency conflicts and to make the installation and replication process more reliable for assessment.

NumPy. NumPy was used for numerical array processing because the software required lightweight handling of coordinates, image-derived data, and intermediate structured values outside the main tensor-training code. It was a natural choice because it integrates well with Python-based scientific and machine-learning workflows.

Pillow. Pillow was used for image loading and resizing because the project processes RGB hand images during training, evaluation, and inference. It was chosen as a simple and dependable library for image I/O and preprocessing without introducing unnecessary complexity.

MediaPipe. MediaPipe was used because the project included an explicit baseline comparison against a strong lightweight hand-tracking system and also relied on MediaPipe-related assets for pseudo-labelled or baseline-oriented workflows. This choice was important because it gave the project a practical reference point against a widely used real-time hand-pose system.

Matplotlib. Matplotlib was used for plotting and qualitative rendering because the project required visual inspection of predictions, generation of qualitative examples, and support for report-ready analysis figures. It was chosen because it is a standard and effective library for producing these outputs in Python.

PyTest. PyTest was used for automated testing because the repository needed a lightweight way to verify important behaviour such as model output shapes, fusion logic, and checkpoint validation. It was chosen to support software quality assurance and to provide a quick validation route for assessors and future users of the artefact.

psutil. The `psutil` package was used in benchmarking workflows because the project included deployment-oriented runtime analysis and needed process-level resource measurements in addition to model accuracy. It was chosen to support practical evaluation of the software on edge-oriented hardware settings.

3.3 Overall Architecture / System Design and Interfaces Between Components

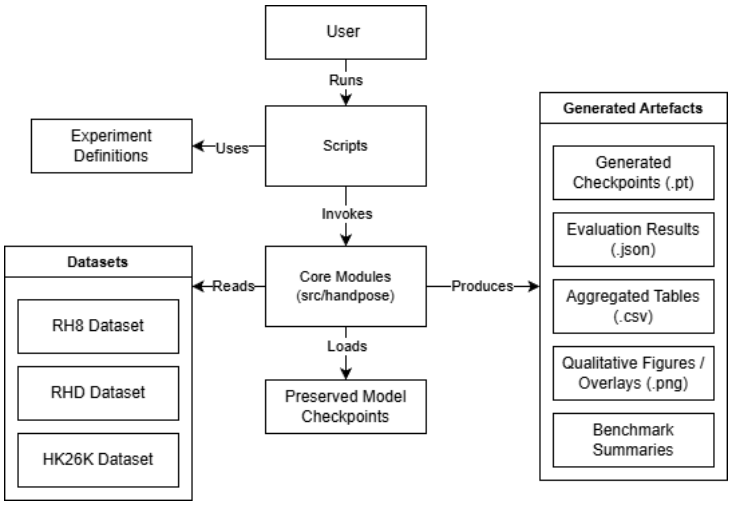

Figure 1: High-level architecture.

Overall Structure

Figure 1 shows the overall system structure. The repository is designed as a modular research artefact in which thin command-line scripts provide the user-facing workflows, while the reusable implementation is contained in the `src/handpose/` package. This separates workflow orchestration from the core hand-pose logic and allows the same internal code to support training, evaluation, benchmarking, and single-image inference.

External datasets and generated artefacts sit at the system boundary, while the internal package provides the common services used by all workflows.

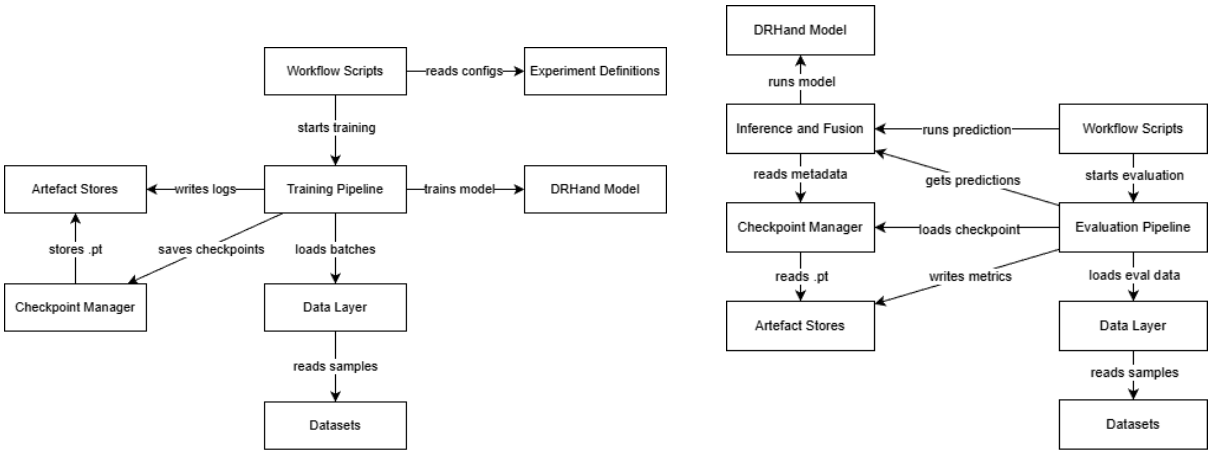

Figure 2: Training and evaluation component views.

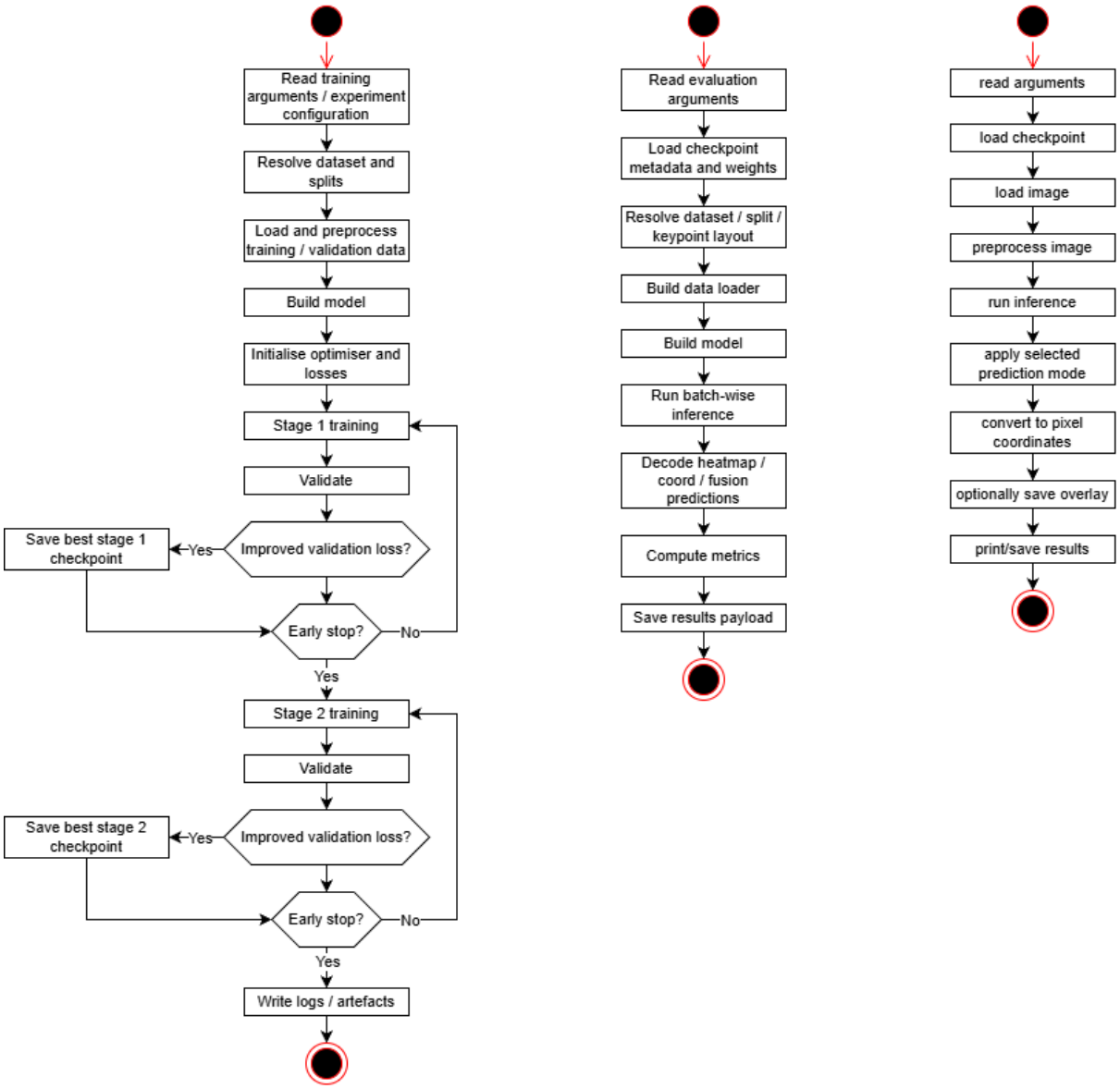

Figure 3: Training, evaluation and inference activity diagrams.

At the highest level, the user interacts with the software by running scripts such as `scripts/train.py`, `scripts/eval_metrics.py`, and `scripts/predict_image.py`. These scripts optionally use experiment definitions and generated command files, then invoke the core modules responsible for data loading, model execution, training, inference, evaluation, and checkpoint handling.

The core modules read dataset files from disk and produce new artefacts such as trained checkpoints, evaluation JSON files, aggregated CSV tables, benchmark summaries, and qualitative figures. This structure keeps the execution interface simple while ensuring that the main implementation remains reusable and consistent across workflows.

Training View

Figure 2 expands this design into separate training and evaluation component views. In the training view, the workflow scripts start the training pipeline, which requests batches from the data layer, passes them through the DRHand model, computes the required losses, and updates the model parameters.

The data layer provides a shared interface for loading and preprocessing supported datasets into normalized tensor representations. During training, the checkpoint manager validates and saves model state together with metadata, while the training pipeline also writes logs and loss histories to the artefact store.

This reflects the repository's two-stage optimisation design, in which the same model is first trained through the heatmap branch and then refined with joint heatmap and coordinate supervision.

Evaluation and Inference View

In the evaluation and inference view, the workflow scripts start either checkpoint evaluation or direct prediction. The evaluation pipeline loads a saved checkpoint through the checkpoint manager, rebuilds the model configuration, obtains evaluation samples from the data layer, and requests predictions from the inference and fusion module.

That module runs the DRHand model, decodes heatmap outputs, uses the coordinate branch predictions, and applies the fusion rule when fused inference is requested.

The resulting predictions are then used by the evaluation pipeline to compute metrics such as SSE, EPE, PCK, timing information, and optional fusion diagnostics, which are written back to the artefact store as structured outputs.

Workflow Activities

Figure 3 complements the component views by showing the workflows as activities rather than static dependencies. It emphasizes the execution order of the main repository tasks: training moves from dataset loading and preprocessing to staged optimisation, validation, and checkpoint export, while evaluation and inference move from checkpoint loading and sample preparation to prediction, fusion, metric computation, and output generation.

This figure is useful because it shows not only which components exist, but also how control and data move through them during actual use.

Interfaces Between Components

The interfaces between these components are mainly file-based and tensor-based. File-based interfaces include dataset roots, annotation files, checkpoint `.pt` files, evaluation `.json` outputs, summary `.csv` files, and rendered `.png` figures.

In-memory interfaces are primarily PyTorch tensors representing images, keypoint coordinates, visibility values, heatmaps, and predicted coordinates. Checkpoint metadata provides an additional interface between workflows by preserving information such as training stage, epoch, keypoint layout, and other configuration details needed to reload and evaluate a model correctly.

Together, these interfaces allow the software to move cleanly from raw dataset input to trained models, reusable checkpoints, and reproducible evaluation outputs.

3.4 Data Model(s), Including In-Memory Structures and Persistent Storage

Processed Data

The main inputs are RGB hand images and per-image annotations from the supported datasets, including 2D keypoint coordinates and visibility values. During preprocessing, these inputs are converted into a shared tensor representation that is reused across training, evaluation, and inference.

In-Memory Representation

The main in-memory structures are image tensors of shape `(C,H,W)`, keypoint-coordinate tensors of shape `(K,2)`, visibility tensors of shape `(K)`, predicted heatmaps of shape `(K,64,64)`, and predicted coordinate tensors of shape `(K,2)`.

This common representation allows the same pipeline to operate with either the standard 21-keypoint layout or the reduced 10-keypoint layout.

Persistent Storage

Persistent storage in the repository is file-based. Source datasets remain on disk as image and annotation files, while intermediate and final outputs are saved as experiment artefacts. Training checkpoints are written as PyTorch `.pt` files containing model state and metadata, loss histories are stored as CSV, evaluation outputs are written as JSON, aggregated summaries are stored as CSV, and qualitative overlays or rendered figures are exported as `.png`.

This design supports reproducibility because trained states, metrics, and generated figures can all be reloaded or compared directly from the filesystem.

3.5 Significant Methods, Functions, or Other Lower-Level Elements

- What lower-level parts of the implementation are most important?
- What does each important method or function do?

Core Model

The most important lower-level implementation element is `HandPoseNet` in `src/handpose/models/hand_pose_`
which acts as the main wrapper around the learned model.

It combines the shared backbone, heatmap regression head, and coordinate regression head into one interface and provides the forward paths used during both training and inference. This makes it the central model component on which the rest of the repository depends.

Data item	In-memory form	Persistent form
RGB input image	Preprocessed tensor `(3,H,W)`	`.png` or `.jpg` image files
Ground-truth hand pose	Keypoint tensor `(K,2)` plus visibility tensor `(K)`	RHD annotations or COCO-style `.json` annotations
Training / evaluation batch	Batched tensors such as `(N,3,H,W)` and `(N,K,2)`	Runtime only; constructed by data loaders
Model predictions	Heatmaps `(N,K,64,64)` and coordinates `(N,K,2)`	Exported prediction or metric payloads in `.json`; optional qualitative `.png` outputs
Checkpoint state	Python dict containing `model_state`, keypoint layout, stage, epoch, and training metadata	PyTorch checkpoint `.pt` files
Result summaries	Scalar metrics and aggregated experiment statistics	CSV tables and JSON result files

Table 1: Main data representations used by the repository.

Architecture and Optimisation

At the architectural level, the most significant lower-level classes are `Backbone`, `Heatmap_reg`, and `coord_reg` in `src/handpose/models/architecture.py`, together with the reusable block definitions in `src/handpose/models/block.py`.

These classes implement the shared feature extractor, the heatmap branch, the coordinate branch, and the depthwise-separable and upsampling building blocks from which the DRHand-inspired model is assembled.

The most important optimisation-related functions are `coords_to_heatmaps`, `HeatmapMSELoss`, and `WingLoss` in `src/handpose/models/losses.py`, along with `build_optimizer_stage1`, `build_optimizer_stage2`, `train_one_epoch`, and `validate` in the training modules.

These functions define how supervision targets are constructed, how the two training losses are computed, how different stages of the model are optimised, and how batch-wise training and validation are carried out.

Data Processing and Fusion

Another important group of lower-level functions appears in the preprocessing and dataset-loading code. The dataset factory and dataset classes in `src/handpose/data/` provide a unified interface for loading RHD and COCO-style hand datasets, while preprocessing helpers in `src/handpose/data/transforms.py` resize images, convert them to tensors, and rescale keypoint coordinates.

These functions are significant because all training, evaluation, and inference workflows depend on the same shared input format.

The lower-level functions implementing branch fusion are also particularly important. In `src/handpose/inference/fusion.py`, functions such as `heatmaps_to_coords_argmax`, `resolve_fusion_bon` and `fuse_coords` define how heatmap predictions are decoded and how the final fused prediction is selected from the heatmap and coordinate branches.

These functions are central because the fusion rule is one of the defining design features of the implemented pipeline.

Checkpointing and Workflow Entry Points

Checkpoint and evaluation handling are also key lower-level parts of the software. Functions in `src/handpose/checkpoints.py` define how checkpoints are validated, saved, and reloaded, while `evaluate_checkpoint` in `src/handpose/evaluation/eval_metrics_core.py` performs the main metric computation for SSE, EPE, PCK, timing, and optional fusion diagnostics.

These components are important because they make the repository reproducible and allow trained models to be reused consistently across training, inference, and result generation workflows.

Finally, the user-facing scripts under `scripts/` are thin but important workflow connectors. Scripts such as `train.py`, `eval_metrics.py`, `predict_image.py`, and `benchmark_pipeline.py` do not contain the main modelling logic themselves, but they are the main entry points through which a user accesses the lower-level implementation for complete training, evaluation, inference, and benchmarking tasks.

3.6 Important Errors and Their Conditions

- What important errors can occur?
- Under what conditions do those errors happen?
- How does the software respond to them?

Input and Configuration Errors

Several important errors in the software are caused by invalid inputs, missing files, or inconsistent configuration between artefacts. For example, the checkpoint utilities raise errors when a checkpoint is missing required metadata fields, has a mismatched keypoint layout, contains duplicate keypoint indices, or uses an unsupported checkpoint version. Similar validation errors occur when invalid keypoint subsets are requested, when unsupported dataset names or split aliases are supplied, or when an invalid prediction mode or negative PCK threshold is passed on the command line.

In these cases, the software responds by raising a clear `ValueError` and stopping execution rather than continuing with inconsistent state.

Runtime Resource Errors

A second group of important errors occurs when required runtime resources are not available. Examples include missing dataset directories, missing image files, empty evaluation image folders, failed imports of optional dependencies such as MediaPipe, or requesting CUDA on a machine where it is not available. These conditions are handled through exceptions such as `FileNotFoundError` or `RuntimeError`.

The software does not silently ignore these problems; instead, it fails early and reports the missing dependency, missing path, or unsupported device condition so that the user can correct the environment before rerunning the workflow.

Workflow State Errors

Further errors can arise from invalid workflow state during training or evaluation. For example, stage-specific training functions raise errors if an unsupported stage number is used, if gradient accumulation is configured with an invalid value, or if stage 2 is requested without the required coordinate loss. Training also raises an error if `-skip-stage1` is used without providing an initial checkpoint.

Evaluation and qualitative-rendering workflows can also fail when no samples match the requested dataset split or when the selected checkpoint does not contain the keypoints required for shared-10 evaluation or fusion. In these situations, the software again responds by terminating with an explicit error message, which helps prevent silent misuse of the repository and makes debugging more straightforward.

3.7 User Interface Design

- How does the user interact with the software?
- What interface choices were made and why?

Command-Line Interaction

The user interacts with the software primarily through a command-line interface. The main workflows are exposed as separate Python scripts under `scripts/`, including training, evaluation, benchmarking, single-image inference, qualitative rendering, and result aggregation.

In practice, the user runs these scripts from the repository root, supplies the required paths and options as command-line arguments, and receives outputs such as checkpoints, JSON result files, CSV summaries, printed metrics, or rendered visualisations.

This command-line design was chosen because the software is a research artefact rather than an end-user application. The main tasks involve running controlled experiments, reproducing reported results, evaluating checkpoints under different conditions, and generating analysis artefacts.

These activities are better supported by explicit script arguments than by a graphical user interface, because command-line workflows are easier to document, automate, reproduce, and integrate with local machines, Conda environments, and HPC or Slurm-based execution.

Separation of Interface and Core Logic

Another important interface choice was to separate the reusable implementation from the user-facing entry points. The core logic is implemented in the `src/handpose/` package, while the scripts act as thin workflow wrappers around that logic.

This was done so that the user interface remains simple at the script level while the lower-level code remains modular and reusable. As a result, users can run complete workflows through a small number of documented commands without needing to interact directly with the internal package structure.

Explicit Runtime Configuration

The interface also relies on explicit file and path arguments for datasets, checkpoints, and output locations. This choice was made to keep the repository portable across different machines and storage layouts, especially since some datasets are external and not bundled with the artefact.

By making these dependencies explicit at runtime, the software avoids hard-coding machine-specific paths and makes the execution process clearer for assessors and future users.

3.8 Quality Assurance Steps

- What steps were taken to improve software quality?
- How was the software checked for correctness, reliability, and maintainability?

Modular Structure

Several steps were taken to improve software quality throughout the project. The repository was organised into a modular structure, with reusable implementation code placed under `src/handpose/` and user-facing workflows exposed through separate scripts.

This separation reduced duplication, made the system easier to understand, and improved maintainability by ensuring that training, evaluation, inference, and benchmarking all relied on shared core logic rather than separate ad hoc implementations.

Validation and Error Handling

Software quality was also improved through explicit validation and defensive error handling. Important interfaces such as checkpoint loading, dataset selection, prediction-mode resolution, keypoint-layout handling, and preprocessing inputs include checks that raise clear exceptions when invalid arguments or inconsistent artefacts are supplied.

This helped reduce silent failure modes and made incorrect usage easier to identify and debug.

Testing and Reproducibility

Correctness and reliability were checked using the lightweight automated test suite under `tests/`. These tests verify important repository contracts such as model output shapes, checkpoint schema validation, and fusion behaviour.

In addition to automated tests, the repository also provides practical validation routes through documented commands in `INSTALL.md` and `REPLICATION_GUIDE.md`, allowing the software to be checked by running real workflows and comparing regenerated outputs against preserved reference artefacts.

Documentation and Artefact Management

Maintainability was further supported through reproducibility-oriented documentation and artefact management. The repository includes installation instructions, requirement notes, a replication guide, curated experiment command files, preserved checkpoints, and saved result artefacts.

These were important quality-assurance measures because they made the software easier to inspect, rerun, and extend, while also ensuring that the reported experimental outputs could be traced back to concrete workflows within the repository.

3.8.1 Version Control and Versioning

- How was version control used during development?
- How were changes tracked and managed?

Version control was used throughout development by maintaining the project as a Git-based repository. This allowed the software, documentation, experiment definitions, and preserved artefacts to be developed within a single structured history rather than as disconnected files.

Using version control was important because the project involved repeated changes to model design, training configuration, evaluation workflows, reporting artefacts, and supporting documentation, all of which needed to remain traceable over time.

Changes were tracked and managed through the normal Git workflow of recording edits to the repository as development progressed. This made it possible to monitor the evolution of

scripts, source modules, experiment files, and paper materials, and helped prevent important changes from being lost or mixed together without record.

It also supported safer experimentation, since alternative implementations or revised configurations could be introduced while preserving the earlier state of the project.

3.8.2 Code Style and Convention

- What code style or conventions were followed?
- How did those conventions help keep the code clear and consistent?

The code follows a consistent Python style based on clear module separation, descriptive function names, and lightweight internal documentation. The repository is organised by responsibility, with separate modules for data loading, model definition, training, inference, evaluation, and checkpoint handling.

Within these modules, functions and classes generally use descriptive snake_case naming for functions and variables, and class-based naming for model components such as `HandPoseNet`. This made the code easier to navigate because related logic was grouped together rather than scattered across large scripts.

A consistent convention was also followed for docstrings and purpose statements. Most modules begin with a short summary docstring explaining their role in the repository, and most important functions include concise docstrings describing what they do.

This helped keep the code clear because users and future readers can understand the role of a file or function without having to infer it entirely from implementation details.

The code also follows a convention of keeping user-facing workflow logic separate from reusable implementation logic. Scripts under `scripts/` mainly parse arguments and call functions from the `src/handpose/` package, while the package itself contains the lower-level logic.

This convention improved consistency because training, evaluation, inference, and benchmarking were all built on shared internal code rather than separate duplicated implementations.

Another important convention is the use of explicit validation and informative error messages at key interfaces. Functions that depend on specific assumptions, such as checkpoint schema, prediction modes, dataset names, keypoint layouts, or device availability, usually check these conditions directly and raise clear exceptions if they are violated.

This kept the behaviour of the software more predictable and made the codebase easier to maintain, since failures tended to occur close to the source of the problem rather than emerging later as ambiguous downstream bugs.

Finally, the repository shows a consistent testing convention in which small focused tests are used to protect important contracts such as model output shapes, checkpoint validation, and fusion behaviour.

This helped reinforce code clarity and consistency because the expected behaviour of core components was made explicit both in implementation and in accompanying tests.

3.8.3 Automated Testing

- What automated tests were implemented?
- What parts of the software do those tests verify?

The repository includes a lightweight automated test suite implemented with `pytest`. The tests are intended as a fast validation layer that checks a small number of important software contracts before running more expensive workflows such as full training or dataset-based evaluation.

This makes them useful both during development and as a first confidence check for assessors or future users of the repository.

One group of tests verifies model-shape behaviour. In `tests/test_model_shapes.py`, the test suite checks that `HandPoseNet` produces outputs of the expected shape for both the standard 21-keypoint configuration and the reduced 10-keypoint configuration.

These tests verify that the model returns heatmaps of shape `(N, K, 64, 64)` and coordinate outputs of shape `(N, K, 2)`, and also confirm that these outputs remain finite. This protects the most basic interface contract of the learned model.

A second group of tests verifies the fusion-related helper functions. In `tests/test_fusion.py`, the software checks that heatmap argmax decoding returns the expected normalized coordinates, that the standard 21-joint fusion edge definition is resolved correctly, and that the reduced tip-and-base 10-joint layout is handled correctly while unsupported layouts are rejected.

These tests verify the correctness of important lower-level logic used in fused inference.

A third group of tests verifies checkpoint schema handling. In `tests/test_checkpoints.py`, the suite checks that valid checkpoints are accepted and normalized correctly, while malformed checkpoints with duplicate keypoint indices or inconsistent keypoint metadata are rejected.

These tests verify the repository's checkpoint contract, which is important because the same checkpoint format is used across training, evaluation, inference, and preserved artefact reuse.

Finally, the `tests/conftest.py` configuration ensures that the `src/` tree is importable when tests are run from the repository root. This supports the automated test workflow by making the repository package available consistently during test execution.

Overall, the automated tests focus on core structural correctness rather than full end-to-end experiment reproduction, while the latter is handled through the documented validation and replication commands elsewhere in the repository.

A Appendices